`scripts/scan-no-hardcoded-uuid.sh` — User Guide

Last verified: 2026-08-26 (§6.J corpus floor — "examined ZERO files" now refuses; LVA vacuous-pass sweep B2) **Inheritance:** HelixConstitution §11.4.18 (script documentation mandate)

Overview

Stub doc generated from the script's in-source header comment. See `scripts/scan-no-hardcoded-uuid.sh` for canonical behavior. This stub exists so the `CM-SCRIPT-DOCS-SYNC` pre-push gate (`.githooks/pre-push` Check 9) starts gating modifications to this script.

The script's own header documentation (verbatim):

```
scripts/scan-no-hardcoded-uuid.sh — standalone §6.R UUID scanner.

Purpose: enforce the §6.R No-Hardcoding Mandate clause that no 36-
char
UUIDs appear in tracked source outside the exemption set.
Extracted as
a standalone script so the hermetic test suite can invoke ONLY
this
rule (without piggy-backing on the broader check-constitution.sh
and
its silent-PASS fall-through bluff). The main checker delegates
here;
tests/check-constitution/test_no_hardcoded_uuid.sh delegates here.

Exit codes:
  0 — no UUID violations
  1 — UUID violation(s) found (paths printed to stderr)

Exemptions (kept in lockstep with the §6.R clause body):
  .env.example                                  — placeholder file
  .lava-ci-evidence/sixth-law-incidents/      — forensic anchors
  docs/superpowers/specs/*.md                 — design docs
  docs/superpowers/plans/*.md                 — implementation
plans
  submodules/                                    — pinned upstream
code
  lava-api-go/third_party/modernc-libc/        — generated/
```

```
vendored upstream code
  *_test.go, *Test.kt, *Tests.kt, *Test.java  — synthetic test
fixtures
```

Usage

See the script's in-source comment block (above) for canonical usage examples.

2026-08-26 update — §6.J corpus floor: an unread corpus is no longer a clean scan

All three §6.R scanners (`scan-no-hardcoded-uuid.sh`, `scan-no-hardcoded-ipv4.sh`, `scan-no-hardcoded-hostport.sh`) received the same change, for the same measured defect. Read this section together with the two sibling docs; the text below is deliberately identical across them because the guard is.

The defect

These scanners are silent on success — a clean scan printed nothing at all and exited 0. The corpus came from `git ls-files -z | grep -zv <exemptions> | while read ...`, and the whole pipeline was terminated with `|| true`. Nothing asserted that a single file had actually been read, so **a broken corpus was byte-identical to a clean scan**:

```
GIT_DIR=/nonexistent/x.git   ->  uuid exit=0   ipv4 exit=0
hostport exit=0
control (normal environment)  ->  uuid exit=0   ipv4 exit=0
hostport exit=0
```

The decisive control: a fixture tree containing real violations of all three rules, in a directory that is **not** a git repository, passed all three scanners. Running `git init` on the same tree — changing nothing about the violating content — made all three fail. The verdict was a function of whether `git ls-files` worked, not of whether the code was clean.

The realistic triggers are ordinary, not exotic: `index.lock` contention from a concurrent git process, a corrupt index, a `safe.directory` dubious-ownership refusal (this checkout lives on an external mount, where that refusal is a known class), or a stray `GIT_DIR` inherited from a parent process. Each one silently

converts §6.R enforcement into a no-op, and — because pre-push runs these through `scripts/check-constitution.sh` — into a green push.

What the scanner now refuses

Three refusals, all **exit 2** (distinct from exit 1, which still means real violations were found in a corpus that was genuinely read):

Condition	Message	Why it is not a pass
`git ls-files` exits non-zero	`6.R SCAN FAILED: could not enumerate the tracked-file corpus.` — git's own stderr is reproduced, indented	The enumeration failed; a clean exit would assert nothing. The message explicitly states this is **not** "no violations found"
The corpus survives the exemption filter but contains no readable regular file	`6.R SCAN FAILED: the scan examined ZERO files.`	A clean exit over an empty corpus asserts nothing, and this scanner prints nothing on success — so the empty case is otherwise indistinguishable from a real pass
Some tracked paths are not readable regular files, are not declared submodule gitlinks, and are not reported deleted by git	`6.R SCAN INCOMPLETE: the scan examined a PARTIAL corpus.` — the offending paths are named	A verdict over a subset asserts nothing about the absent files. A floor that fires only at exactly zero is a floor with one stair

Each message distinguishes its cause rather than emitting one generic failure. The zero-files refusal, for example, separates "*`git ls-files` succeeded but the exemption filter left nothing*" (wrong tree, or the filter regex has drifted to exclude everything) from "*paths were listed but none is a readable regular file*" (working-tree drift, not an empty repository), because those two have different remedies and a diagnosis that misstates its cause sends the reader to the wrong one.

Why the expectation is derived, never hardcoded

The floor compares against `git ls-files` output and `.gitmodules`, not against a literal file count. A hardcoded count goes stale on the next commit that adds or removes a file, and a stale floor is the same defect wearing a different mask.

Two classes of tracked path are legitimately **not** readable regular files, and both are classified rather than dropped:

- **Declared submodule gitlinks** — read from `.gitmodules` (`path = ...`). Expected; contributes nothing to scan and triggers nothing.
- **Paths git itself reports deleted** (`git ls-files --deleted`) — a pending `git rm` or an unstaged deletion. Already visible to the operator through `git status`, and a deleted file holds no content that could carry a violation. These are **excluded from the expectation** and **named in the clean verdict**, so the scanner's claim always states the corpus it actually read.

The distinction that carries the weight: *absent and reported by git* is a deletion in progress; *absent and not reported by git* is invisible drift that no other gate would surface. Only the second is a refusal.

The clean verdict now states its corpus

Silence on success was half the defect, so success now says what it examined (written to stderr, so stdout-consuming callers are unaffected):

```
6.R UUID scan clean: <N> tracked file(s) examined.
```

When pending deletions were excluded, the count is followed by a parenthetical naming how many and why.

Exit codes (updated)

The header block quoted above lists only 0 and 1; it predates this change and is incomplete. The full set is:

Code	Meaning
0	No violations — **and** at least one file was actually examined
1	Violation(s) found (paths printed to stderr)
2	The scan could not be trusted: corpus enumeration failed, examined zero files, or examined a partial corpus

Callers that previously treated any non-zero exit as "violations found" now distinguish 1 (a real finding) from 2 (a scan that must not be believed either way).

§6.J falsifiability rehearsal

1. `GIT_DIR=/nonexistent/x.git bash scripts/scan-no-hardcoded-uuid.sh` → expect **exit 2** with `could not enumerate the tracked-file corpus` and git's own stderr reproduced. Before this change: exit 0, no output.
2. Run normally in the Lava tree → expect **exit 0** and the corpus-count line.
3. `git rm --cached <some tracked source file>` (leaving the file on disk), re-run → still exit 0; the file simply leaves the corpus, and the reported count drops.

Maintenance

When this script is modified, update this document in the same commit (CM-SCRIPT-DOCS-SYNC requires it). Per §11.4.18, the documentation MUST stay in sync with the codebase — no doc may be out of sync with its script.

Cross-references

- `scripts/scan-no-hardcoded-uuid.sh` — the script itself
- `docs/helix-constitution-gates.md` — gate inventory
- HelixConstitution `Constitution.md` §11.4.18 (the mandate)
- Lava `CLAUDE.md` §6.AD (HelixConstitution Inheritance)